MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1

Software Design

Author: Ilie-Mihai, Morovan
Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

03/2026

Contents

1 Problem definition **2**
2 Analysis phase - Use cases & requirements **2**
2.1 Functional requirements 2
2.2 Use cases 2
3 Design phase **4**
3.1 Architecture overview 4
3.2 Entity-relationship diagram 4
3.3 Authentication Component 5
3.4 Movie and WatchListManagement 6
3.5 Design patterns 6
3.6 Class diagrams 6
3.7 Sequence diagrams 7
4 Implementation phase - Application **8**
4.1 Tools and technologies 8
4.2 Application structure 9
4.3 User manual 9
5 Testing and validation **10**

1 Problem definition

The objective of this project is to design and implement a movie management application using a layered architecture. The system allows users to browse movies, search and filter them, and manage personal watchlists.

The application must support multiple user roles with different permissions. Visitors can only view data, authenticated users can manage their watchlist, and administrators have full control over movie data.

The project follows software design principles such as separation of concerns, modularity, and maintainability.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

The system must implement the following functionalities:

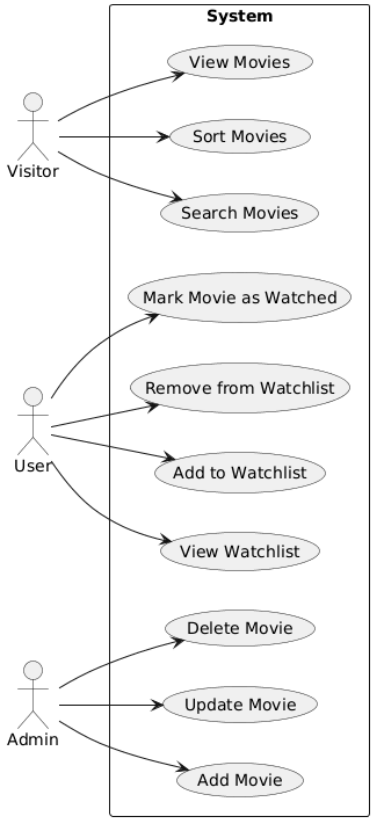
-User authentication (login with username and password)
-Role-based access control (Visitor, User, Admin)
-View list of movies
Search movies by title
Filter movies by genre
Sort movies (ascending/descending)
Add, update, and delete movies (Admin only)
Manage watchlist:
    Add movie to watchlist
    Remove movie from watchlist
    Mark movie as watched

2.2 Use cases

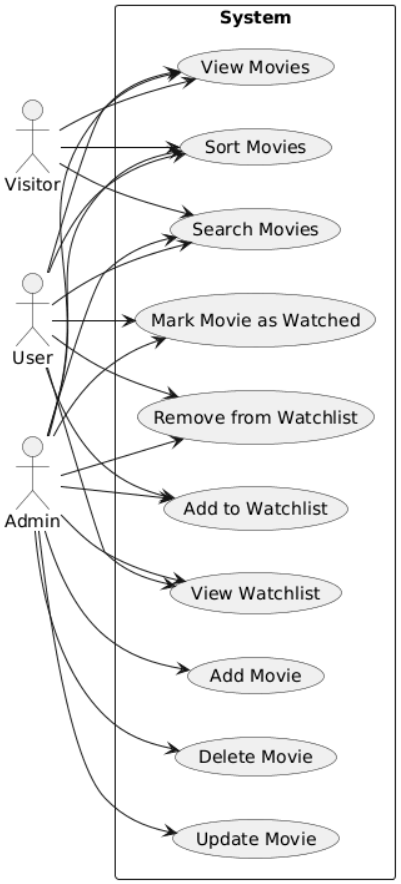
Visitor: View movies
Sort/filter movies
Search movies

User: Login into the system
View movies
Search movies
Filter and sort movies
Add movies to watchlist
Remove movies from watchlist
Mark movies as watched

Admin: All user cases
Login into the system
Perform all user actions
Add movies
Update movies
Delete movies



Pretty Use Case Diagram



Full UseCase Diagram

3 Design phase

3.1 Architecture overview

The application follows a Model-View-Controller (MVC) architectural pattern, which separates the system into three main layers:

Model: Contains entities (User, Movie, WatchlistItem) and repositories that manage database access.
View: Implemented using JavaFX with FXML files to structure the UI.
Controller: Handles user interactions, communicates with the service layer, and updates the view.

A Service layer is added to encapsulate business logic, enforce access control, and mediate between controllers and repositories.

Architectural Style: Layered architecture (MVC is a type of layered design)

Advantages:

Clear separation of concerns
Easier to maintain and extend
Facilitates unit testing and debugging

Disadvantages:

Slight overhead due to multiple layers
Requires careful coordination between layers

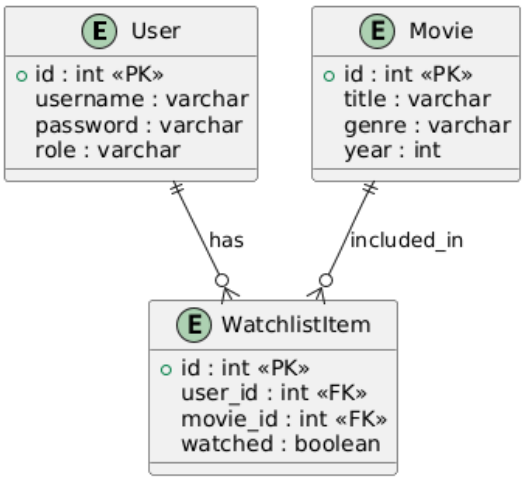
3.2 Entity-relationship diagram

Table Descriptions

1. User Table
   - Holds all registered users.
   - id is primary key.
   - One user can have multiple watchlist items.
2. Movie Table
   - Holds all movies in the system.
   - id is primary key.
   - Movies can appear in many users' watchlists.
3. WatchlistItem Table
   - Tracks which movies are added by which users and their watched status.
   - user_id and movie_id are foreign keys referencing User and Movie.
   - Represents a many-to-many relationship between User and Movie with additional attribute watched.

Relationships:

A user can have multiple watchlist items
A movie can appear in multiple watchlist items



3.3 Authentication Component

This component handles user authentication and session management.

Classes Involved: AuthService, UserRepository, LoginController

Responsibilities: Validate user credentials, Retrieve user data from the database, Return authenticated user to the system

Interactions:

LoginController receives login requests and calls AuthService.
AuthService retrieves user data from UserRepository.
UserRepository queries the database for the specified username.
AuthService verifies the password and returns success or failure.

3.4 Movie and WatchListManagement

This component manages movies and user watchlists. It includes:

MovieService
WatchlistService
MovieRepository
WatchlistRepository
MovieController

Responsibilities:

CRUD operations for movies (Admin only)
Search, filter, and sort movies
Manage watchlist operations
Enforce role-based permissions

3.5 Design patterns

The following design patterns are used:

MVC (Model-View-Controller)

Category: Architectural/structural
Problem it solves: Separates user interface, business logic, and data management
Classes involved: MovieController, LoginController, AuthService, MovieService, Repositories

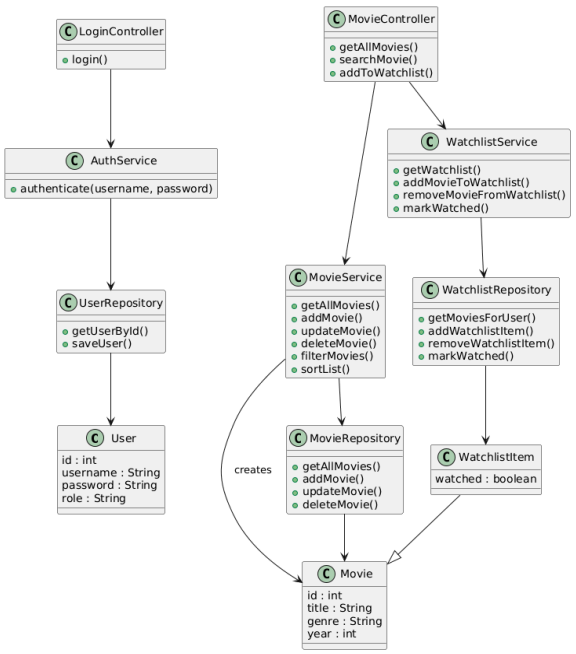
Repository Pattern

Category: Structural
Problem it solves: Abstracts data access logic and provides a unified interface to the database
Classes involved: UserRepository, MovieRepository, WatchlistRepository

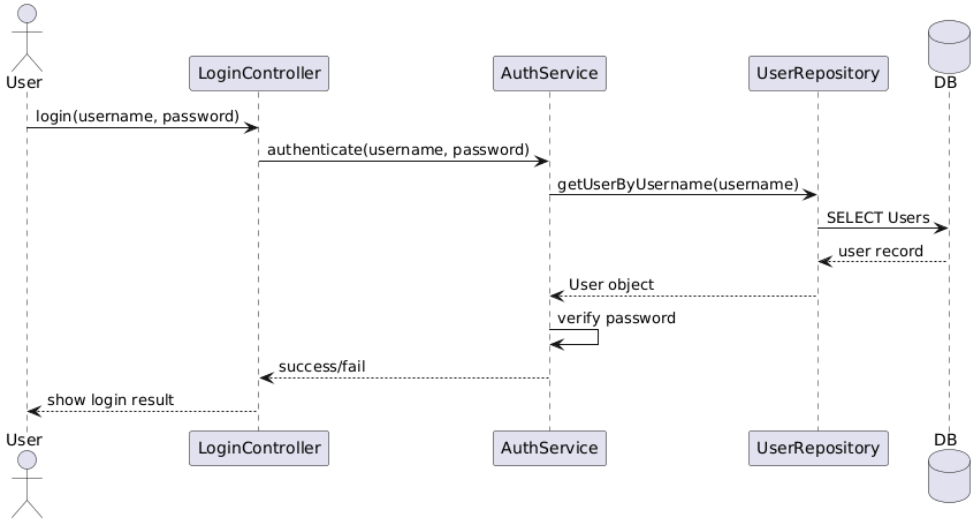
Service Layer Pattern

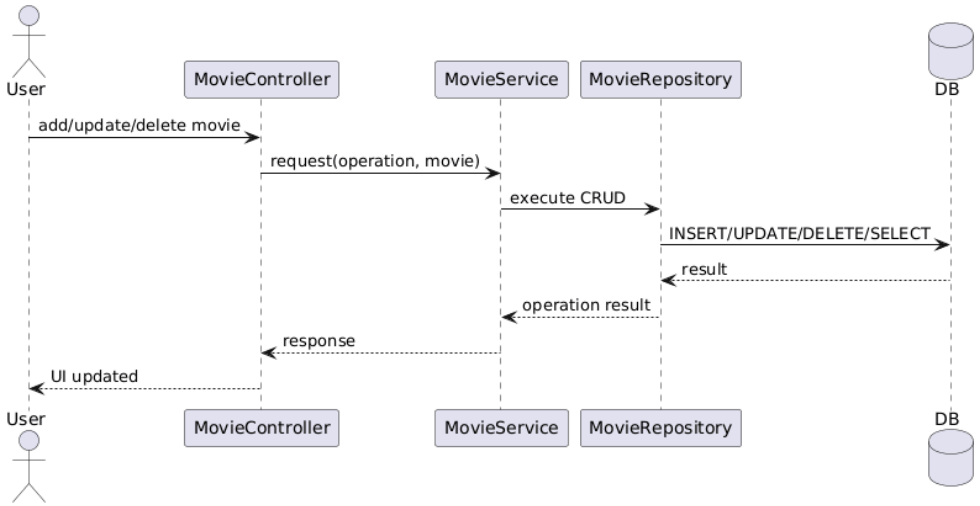
Category: Structural
Problem it solves: Encapsulates business logic and ensures that controllers remain thin
Classes involved: AuthService, MovieService, WatchlistService

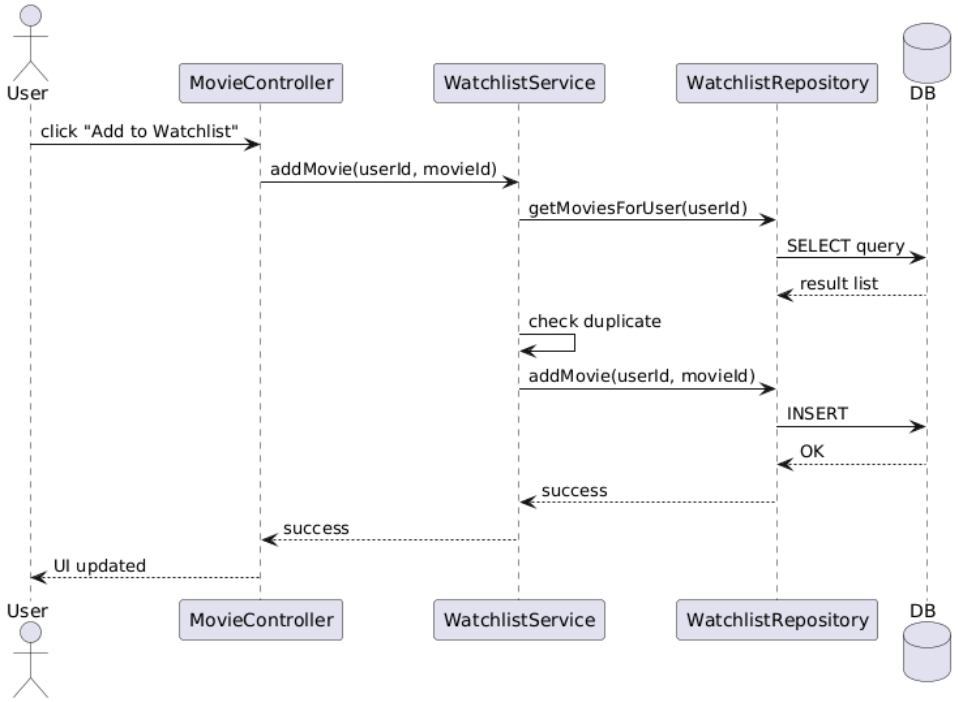
3.6 Class diagrams



3.7 Sequence diagrams







4 Implementation phase - Application

4.1 Tools and technologies

This subsection presents the technology stack used to implement the application:

Programming Language: Java
UI Framework: JavaFX with FXML
Database: PostreSQL
Design Approach: Object-Oriented Programming, MVC architecture
Communication: Local method calls (no remote API)

Reasons for choosing these technologies:

Java: Platform-independent and supports OOP
JavaFX + FXML: Simplifies UI creation and separation from logic
PostreSQL: Simple and widely supported relational database
MVC: Clear separation of concerns for maintainability

4.2 Application structure

The application is structured into the following layers:

Controller Layer: Handles user input and UI events

Service Layer: Contains business logic; Enforces role-based access control

Repository Layer: Manages data storage

Model Layer: Defines entities

4.3 User manual

Login:

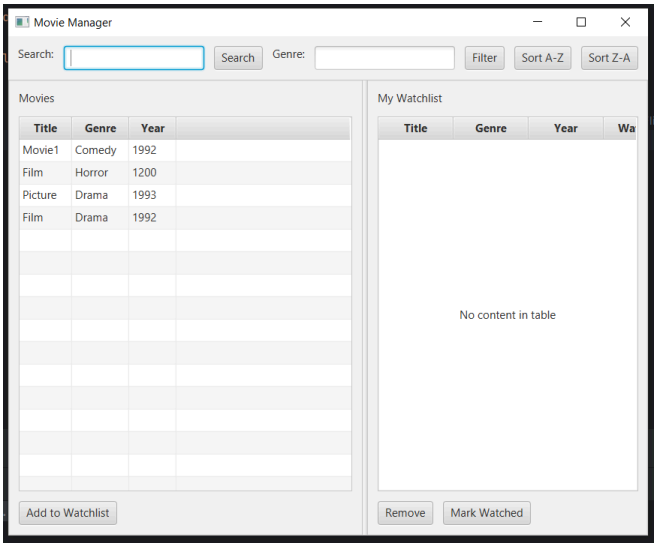
The user enters username and password
The system authenticates and assigns a role

Visitor:

Can only view movies

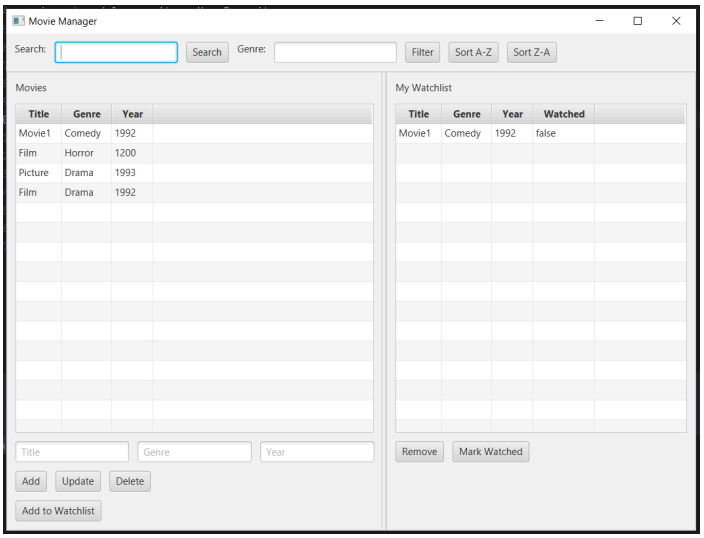
User:

Can search, filter, and sort movies
Can manage watchlist



Admin:

Can add, update, and delete movies
Has full access to all features



5 Testing and validation

The application was tested manually using different user roles:

Visitor
Verified that no modification actions are allowed
User
Verified watchlist functionality
Verified search and filtering
Admin
Verified CRUD operations on movies

All functionalities behaved as expected, and role-based restrictions were correctly enforced both in the UI and service layer.